

ĐẠI HỌC XÂY DỰNG HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN

—o0o—



ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC

NGÀNH: CÔNG NGHỆ THÔNG TIN

*Phát triển hệ thống blockchain riêng biệt cho một chuỗi
doanh nghiệp (khoai-chain)*

Người Thực hiện:

Cán Tất Dương

Hà Nội, năm 2026

Mục lục

Lời nói đầu	3
1 Giới thiệu đề tài	4
1.1 Lý do chọn đề tài	4
1.1.1 Sự phức tạp và công kênh của các nền tảng blockchain hiện tại . . .	4
1.1.2 Nhu cầu về một hệ thống blockchain thử nghiệm	4
1.1.3 Nhu cầu thực tiễn của doanh nghiệp vừa và nhỏ	4
1.2 Mục tiêu nghiên cứu	4
1.2.1 Tối ưu hóa quá trình triển khai và khởi chạy node	4
1.2.2 Xây dựng lõi blockchain chuyên biệt hóa cho chuỗi doanh nghiệp . .	5
1.3 Đối tượng và phạm vi nghiên cứu	5
2 Cơ sở lý thuyết và Công nghệ nền tảng	6
2.1 Tổng quan về công nghệ Blockchain	6
2.1.1 Khái niệm và nguyên lý hoạt động cơ bản	6
2.1.2 Các đặc trưng cốt lõi của Blockchain	6
2.1.3 Phân loại mạng lưới Blockchain	7
2.2 Cơ sở mật mã học trong Blockchain	7
2.2.1 Hàm băm mật mã (Cryptographic Hash Function)	7
2.2.2 Chữ ký số (Digital Signatures) và Mã hóa bất đối xứng	7
2.3 Mạng ngang hàng (Peer-to-Peer Network)	8
2.3.1 Kiến trúc Client-Server so với P2P	8
2.3.2 Giao thức truyền bá thông điệp (Gossip Protocol)	8
2.4 Cơ chế đồng thuận (Consensus Mechanisms)	8
2.4.1 Bài toán Tướng Byzantine (Byzantine Generals Problem)	8
2.4.2 Các thuật toán đồng thuận truyền thống (PoW, PoS)	9
2.4.3 Đồng thuận trong Private Blockchain (CFT và BFT)	9
2.5 Hợp đồng thông minh (Smart Contract)	10
2.5.1 Khái niệm và đặc tính tất định (Determinism)	10
2.5.2 Vòng đời thực thi của Hợp đồng thông minh	10
2.6 Phân tích kiến trúc nền tảng Hyperledger Fabric	10
2.6.1 Mô hình Execute-Order-Validate	10
2.6.2 Vấn đề thực tiễn và bài toán tối ưu tài nguyên	11
3 Phân tích và Đặc tả yêu cầu hệ thống	12
3.1 Kiến trúc tổng quát của Khoai Chain	12
3.1.1 Các thay đổi chính	12
3.1.2 So sánh với Hyperledger Fabric	13
3.2 Mô hình các chức năng	14
3.3 Các trường hợp sử dụng	20
3.4 Yêu cầu phi chức năng	25
4 Thiết kế Kiến trúc Khoai Chain	26
4.1 Kiến trúc tổng thể hệ thống	26
4.2 Thiết kế Module P2P (Peer, Server, Handler, Message)	26
4.3 Thiết kế Core Blockchain	26
4.3.1 Cấu trúc dữ liệu Block và Transaction	26
4.3.2 Luồng hoạt động của Mempool	26

4.4	Thiết kế hệ thống Smart Contract (Manager, Router, Interface)	26
4.5	Thiết kế Cấu trúc lưu trữ dữ liệu (Database Storage)	26
5	Hướng dẫn sử dụng và Triển khai	27
5.1	Khởi tạo môi trường và cấu hình (khoai-config.yaml, get-env)	27
5.2	Chạy thử nghiệm bằng CLI và Docker	27
5.3	Kịch bản chạy thử nghiệm mẫu cho sinh viên	27
5.4	Các chức năng đang phát triển và Hạn chế hiện tại	27
Kết luận		28
	Tóm tắt các chức năng đã chốt	28
	Định hướng hoàn thiện mã nguồn	28

Lời nói đầu

Chương 1: Giới thiệu đề tài

1.1. Lý do chọn đề tài

1.1.1. Sự phức tạp và công kênh của các nền tảng blockchain hiện tại

Từ thực tiễn triển khai các bài toán nghiệp vụ phức tạp, điển hình như hệ thống quản lý chuỗi cung ứng thuốc trên nền tảng như Hyperledger Fabric, công nghệ này bộc lộ rõ những rào cản lớn về mặt kỹ thuật và vận hành. Cụ thể, kiến trúc của Fabric đòi hỏi tài nguyên phần cứng đồ sộ để duy trì các thành phần độc lập (như Orderer, Peer, CA) và yêu cầu thiết lập hệ thống chứng chỉ số phức tạp. Quá trình cấu hình, gỡ lỗi và triển khai mạng lưới tốn rất nhiều thời gian, trở thành một nút thắt lớn (bottleneck) cho các nhà phát triển. Sự công kênh này hoàn toàn không phù hợp với các mô hình thử nghiệm nhanh, các dự án giáo dục, hoặc các doanh nghiệp có hạ tầng hạn chế, từ đó đặt ra yêu cầu cấp thiết về một nền tảng blockchain tinh gọn hơn nhưng vẫn đảm bảo tính bất biến của dữ liệu.

1.1.2. Nhu cầu về một hệ thống blockchain thử nghiệm

Từ góc độ nghiên cứu và giáo dục, việc tiếp cận kiến trúc blockchain doanh nghiệp hiện nay gặp rất nhiều trở ngại do yêu cầu khắt khe về phần cứng. Người học và nhà phát triển thiếu đi một môi trường thử nghiệm tinh gọn, nơi họ có thể nhanh chóng khởi tạo một mạng lưới blockchain độc lập ngay trên máy tính cá nhân. Nhu cầu thực tế đòi hỏi một hệ thống được đóng gói nhẹ nhàng, đi kèm với các bộ công cụ phát triển (SDK) được tách biệt rõ ràng. Điều này giúp sinh viên và lập trình viên tập trung hoàn toàn vào việc xây dựng logic nghiệp vụ thông qua Hợp đồng thông minh (Smart Contract), thay vì tốn thời gian xử lý các cấu hình mạng ngang hàng (P2P) hay cơ chế đồng thuận ở tầng lõi.

1.1.3. Nhu cầu thực tiễn của doanh nghiệp vừa và nhỏ

Các doanh nghiệp vừa và nhỏ (SME) hoặc các chuỗi cung ứng quy mô nội bộ luôn có nhu cầu ứng dụng blockchain để giải quyết bài toán minh bạch hóa dữ liệu, chống giả mạo thông tin và truy xuất nguồn gốc. Mặc dù vậy, bài toán chi phí hạ tầng của các giải pháp chuẩn công nghiệp thường vượt quá khả năng đáp ứng của họ. Nhóm doanh nghiệp này cần một giải pháp blockchain được thiết kế tối giản nhưng vẫn giữ nguyên các đặc tính cốt lõi của sổ cái phân tán. Việc phát triển một hệ thống blockchain riêng biệt, có khả năng "cắm và chạy" (plug-and-play), tiêu tốn ít tài nguyên và dễ dàng tích hợp vào cơ sở hạ tầng máy chủ hiện có, sẽ giải quyết triệt để rào cản chi phí đồng thời bảo vệ an toàn cho các dữ liệu nhạy cảm của doanh nghiệp.

1.2. Mục tiêu nghiên cứu

Đề tài hướng tới việc giải quyết những rào cản kỹ thuật của các nền tảng sổ cái phân tán truyền thống bằng cách xây dựng một hệ thống tinh gọn. Mục tiêu tổng thể được chia thành hai khía cạnh chính nhằm phục vụ cả môi trường nghiên cứu học thuật lẫn ứng dụng thực tiễn.

1.2.1. Tối ưu hóa quá trình triển khai và khởi chạy node

Hệ thống được thiết kế để giảm thiểu tối đa các bước thiết lập môi trường phức tạp. Bằng cách phát triển các công cụ giao diện dòng lệnh (CLI framework) và đóng gói toàn

bộ quy trình khởi chạy dưới dạng các container, mục tiêu là cho phép người dùng có thể kích hoạt một node mạng lưới độc lập chỉ bằng vài thao tác cơ bản. Quá trình triển khai tự động này loại bỏ sự phụ thuộc vào các dịch vụ hạ tầng trung gian, giúp việc kết nối vào mạng ngang hàng (P2P), thử nghiệm cục bộ và gỡ lỗi diễn ra nhanh chóng. Điều này đặc biệt hữu ích trong việc tiết kiệm tài nguyên tính toán, cho phép chạy mượt mà ngay trên các thiết bị máy tính cá nhân tiêu chuẩn.

1.2.2. Xây dựng lõi blockchain chuyên biệt hóa cho chuỗi doanh nghiệp

Mục tiêu trọng tâm là phát triển một lõi (core blockchain) được tinh chỉnh trực tiếp cho các luồng nghiệp vụ nội bộ. Kiến trúc lõi sẽ tự chủ hoàn toàn các thành phần như quản lý mempool, đồng bộ block, giao thức P2P và định tuyến giao dịch (contract router). Bằng cách cung cấp một môi trường thực thi được cô lập và tách biệt qua bộ SDK, các nhà phát triển có thể trực tiếp ánh xạ các quy tắc kinh doanh thành Hợp đồng thông minh mà không làm ảnh hưởng đến tính ổn định của toàn mạng. Khác với kiến trúc phục vụ đám đông, hệ thống lõi này chuyên biệt hóa cho một nhóm các định chế hoặc đối tác kinh doanh đã được xác thực, đảm bảo giao dịch được lưu vết minh bạch, nhất quán nhưng vẫn nằm trong phạm vi giới hạn an toàn của chuỗi.

1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài là công nghệ sổ cái phân tán (Distributed Ledger Technology), mạng ngang hàng (P2P), và cơ chế thực thi Hợp đồng thông minh (Smart Contract).

Phạm vi nghiên cứu tập trung vào việc thiết kế và xây dựng lõi hệ thống Khoai Chain từ đầu, giới hạn trong môi trường mạng nội bộ (private/consortium) phục vụ chuỗi doanh nghiệp vừa và nhỏ hoặc mục đích giáo dục. Đề tài không tập trung giải quyết các bài toán của public blockchain như phát hành tiền mã hóa (tokenomics), khai thác (mining), hay khả năng mở rộng hàng triệu người dùng.

Chương 2: Cơ sở lý thuyết và Công nghệ nền tảng

2.1. Tổng quan về công nghệ Blockchain

2.1.1. Khái niệm và nguyên lý hoạt động cơ bản

Blockchain là một công nghệ sổ cái phân tán (Distributed Ledger Technology - DLT) cho phép ghi lại các giao dịch một cách an toàn, minh bạch và không thể đảo ngược trên một mạng lưới máy tính ngang hàng (P2P). Thay vì lưu trữ dữ liệu tại một máy chủ trung tâm, dữ liệu của blockchain được nhân bản và đồng bộ hóa tại tất cả các nút (nodes) tham gia mạng lưới.

Cấu trúc lưu trữ dữ liệu của Blockchain bao gồm chuỗi các khối (Blocks) liên kết tuyến tính với nhau theo thời gian. Mỗi khối chứa hai thành phần chính:

- **Block Header (Tiêu đề khối):** Chứa siêu dữ liệu (metadata) quản lý khối, bao gồm:
 - + **Version:** Phiên bản của giao thức blockchain.
 - + **Previous Hash:** Mã băm của toàn bộ header khối trước đó. Đây là sợi xích toán học liên kết các khối. Nếu dữ liệu khối trước bị sửa đổi, mã băm này thay đổi và làm gãy toàn bộ chuỗi phía sau.
 - + **Merkle Root Hash:** Mã băm gốc của cây Merkle, đại diện cho toàn bộ giao dịch trong khối.
 - + **Timestamp:** Thời gian khối được tạo ra.
 - + **Nonce / Bits:** Các tham số phục vụ cho cơ chế đồng thuận (ví dụ trong PoW).
- **Block Body (Thân khối):** Chứa danh sách chi tiết các giao dịch thực tế đã được xác thực và đóng gói vào khối.

2.1.2. Các đặc trưng cốt lõi của Blockchain

- **Tính phân tán (Decentralization):** Hệ thống loại bỏ thực thể trung tâm quản lý. Quyền lực ghi và xác thực sổ cái được chia đều cho các nút trong mạng theo luật đồng thuận, giúp loại bỏ điểm lỗi duy nhất (Single Point of Failure - SPOF).
- **Tính minh bạch (Transparency):** Mọi tài khoản tham gia mạng lưới (đặc biệt là Public) đều có quyền truy cập, tra cứu và kiểm toán toàn bộ lịch sử giao dịch từ khối khởi thủy (Genesis Block) đến khối hiện tại.
- **Tính bất biến (Immutability):** Dữ liệu một khi đã được đồng thuận ghi vào chuỗi thì không thể sửa đổi hoặc xóa bỏ. Muốn thay đổi một khối, kẻ tấn công phải thay đổi tất cả các khối sau nó và chiếm trên 51% sức mạnh toán học hoặc quyền kiểm soát mạng lưới (điều này bất khả thi về mặt chi phí trong các mạng lớn).
- **Khả năng truy xuất (Traceability):** Nhờ cơ chế liên kết mã băm và mốc thời gian (timestamp) cố định, mọi thực thể dữ liệu hoặc tài sản số hóa lưu trữ trên blockchain đều có thể được truy vết nguồn gốc dịch chuyển một cách chính xác qua từng địa chỉ ví.

2.1.3. Phân loại mạng lưới Blockchain

- **Public Blockchain (Mạng công khai):** Bất kỳ ai cũng có thể tham gia đọc, ghi dữ liệu và tham gia vào quá trình đồng thuận (ví dụ: Bitcoin, Ethereum). Mạng này có tính phi tập trung tối đa nhưng tốc độ xử lý giao dịch (TPS) thấp và chi phí vận hành lớn.
- **Private Blockchain (Mạng tư nhân):** Quyền đọc, ghi và đồng thuận hoàn toàn thuộc về một tổ chức duy nhất quản lý. Tốc độ xử lý rất nhanh, bảo mật nội bộ cao, nhưng mang tính tập trung cao.
- **Consortium Blockchain (Mạng liên minh):** Quyền kiểm soát mạng lưới được chia sẻ cho một nhóm các tổ chức hoặc doanh nghiệp có lợi ích chung (ví dụ: liên minh ngân hàng, chuỗi cung ứng).

Mối liên hệ với đề tài: Đối với các bài toán chuỗi doanh nghiệp, Consortium/Private Blockchain (như Hyperledger Fabric) là mô hình tối ưu nhất. Nó vừa giữ được tính minh bạch, bất biến của blockchain giữa các bên đối tác, vừa giải quyết được bài toán bảo mật dữ liệu kinh doanh, kiểm soát quyền truy cập nghiêm ngặt và đạt hiệu năng xử lý cao (TPS lớn, độ trễ thấp) đáp ứng quy mô công nghiệp.

2.2. Cơ sở mật mã học trong Blockchain

2.2.1. Hàm băm mật mã (Cryptographic Hash Function)

Hàm băm mật mã là một thuật toán toán học chuyển đổi một đầu vào dữ liệu có kích thước bất kỳ thành một chuỗi ký tự đầu ra có độ dài cố định. Trong các hệ thống blockchain, thuật toán SHA-256 (Secure Hash Algorithm 256-bit, dùng trong Bitcoin) hoặc Keccak256 (dùng trong Ethereum) được áp dụng phổ biến. Một hàm băm mật mã học tiêu chuẩn bắt buộc phải đảm bảo 3 đặc tính cốt lõi sau:

- **Tính tất định (Determinism):** Với cùng một thông điệp đầu vào M , hàm băm luôn luôn trả về một kết quả đầu ra $H(M)$ duy nhất, không thay đổi qua các lần tính toán.
- **Tính kháng tiền ảnh (Pre-image resistance / Hàm một chiều):** Khi biết trước một giá trị băm Y , về mặt toán học cực kỳ khó khăn (bất khả thi trong thời gian thực) để tìm lại thông điệp gốc X sao cho $H(X) = Y$. Cách duy nhất là thử sai ngẫu nhiên (Brute-force).
- **Tính kháng va chạm (Collision resistance):** Cực kỳ khó để tìm thấy hai thông điệp khác nhau hoàn toàn X_1 và X_2 mà khi băm lại cho ra cùng một kết quả ($H(X_1) = H(X_2)$). Ngoài ra, hàm băm có hiệu ứng tuyết lở (Avalanche effect): chỉ cần thay đổi 1 bit ở đầu vào, toàn bộ chuỗi đầu ra sẽ thay đổi hoàn toàn.

2.2.2. Chữ ký số (Digital Signatures) và Mã hóa bất đối xứng

Mã hóa bất đối xứng sử dụng một cặp khóa toán học liên kết chặt chẽ với nhau: Khóa bí mật (Private Key) dùng để tạo chữ ký số và Khóa công khai (Public Key) dùng để xác thực chữ ký hoặc định danh địa chỉ ví.

Trong Blockchain, thuật toán ECDSA (Elliptic Curve Digital Signature Algorithm) - Thuật toán chữ ký số dựa trên đường cong elliptic - được áp dụng để tối ưu hóa kích thước khóa và tốc độ tính toán so với RSA truyền thống.

- **Quy trình ký (Signing):** Khi người dùng muốn thực hiện giao dịch, họ dùng Private Key của mình kết hợp với mã băm của nội dung giao dịch thông qua thuật toán ECDSA để tạo ra một chuỗi chữ ký số (*Sig*).
- **Quy trình xác thực (Verification):** Các nút trong mạng nhận được giao dịch sẽ dùng Public Key của người gửi (được công khai) để giải mã và kiểm tra tính hợp lệ của *Sig*. Quy trình này chứng minh người gửi sở hữu tài khoản và đồng ý chuyển tiền mà không cần tiết lộ Private Key cho mạng lưới, ngăn chặn hoàn toàn việc giả mạo giao dịch.

2.3. Mạng ngang hàng (Peer-to-Peer Network)

2.3.1. Kiến trúc Client-Server so với P2P

- **Kiến trúc Client-Server:** Dữ liệu tập trung hoàn toàn tại hệ thống máy chủ (Server). Các máy trạm (Client) muốn tương tác đều phải gửi yêu cầu về Server. Kiến trúc này chịu nhược điểm SPOF (Single Point of Failure): nếu máy chủ bị sập, bị tấn công DDoS, hoặc bị hack dữ liệu, toàn bộ hệ thống sẽ tê liệt và dữ liệu có nguy cơ bị thao túng hoàn toàn.
- **Kiến trúc P2P (Peer-to-Peer):** Mạng lưới bao gồm các nút (Peer) có vai trò và quyền hạn ngang hàng nhau. Mỗi nút vừa hoạt động như một Client (yêu cầu dữ liệu) vừa như một Server (cung cấp, lưu trữ, xác thực dữ liệu). Khi một số nút bị ngắt kết nối hoặc bị tấn công, hệ thống vẫn vận hành bình thường nhờ các bản sao tại các nút còn lại, loại bỏ hoàn toàn bài toán SPOF.

2.3.2. Giao thức truyền bá thông điệp (Gossip Protocol)

Gossip Protocol (Giao thức dịch bệnh/tin đồn) là cơ chế giao tiếp chính giúp các nút trong mạng lưới P2P phi tập trung có thể phát tán thông tin (như giao dịch mới hoặc khối mới tạo) ra toàn mạng một cách nhanh chóng và tiết kiệm băng thông.

- **Nguyên lý hoạt động:** Khi một nút nhận được một thông điệp mới, nó sẽ chọn ngẫu nhiên một số lượng nút K hàng xóm (Fanout) để chuyển tiếp thông điệp đó. Các nút nhận tiếp theo lại lặp lại quy trình chọn ngẫu nhiên này.
- **Hiệu quả:** Thông tin được lan truyền theo cấp số nhân (Exponential growth). Chỉ trong một thời gian cực ngắn với độ phức tạp thời gian là $O(\log N)$ (với N là tổng số nút), toàn bộ mạng lưới rộng lớn sẽ đạt được trạng thái đồng bộ thông tin mà không cần một máy chủ điều phối trung tâm điều hướng luồng tin.

2.4. Cơ chế đồng thuận (Consensus Mechanisms)

2.4.1. Bài toán Tướng Byzantine (Byzantine Generals Problem)

Được đặt ra bởi Lamport, Shostak và Pease vào năm 1982, bài toán mô tả một nhóm các vị tướng Byzantine vây hãm một thành phố địch. Họ liên lạc với nhau chỉ bằng tin nhắn messenger. Các tướng phải đồng thuận về một kế hoạch duy nhất: cùng Tấn công hoặc cùng Rút lui. Nếu có một vài tướng phản bội (hoặc tin nhắn bị thất lạc), họ sẽ cố tình gửi các thông điệp mâu thuẫn (gửi "Tấn công" cho tướng này nhưng gửi "Rút lui" cho tướng khác) nhằm làm chia rẽ và thất bại quân đội.

Ý nghĩa trong Blockchain: Đây là mô hình hóa thách thức đạt được sự đồng thuận dữ liệu trong một hệ thống phân tán, nơi các nút tham gia mạng có thể bị lỗi kỹ thuật (crash), mất kết nối, hoặc tệ hơn là chủ động gian lận, gửi dữ liệu sai lệch để trục lợi.

2.4.2. Các thuật toán đồng thuận truyền thống (PoW, PoS)

- **Proof of Work (PoW - Bằng chứng công việc):** Các nút (thợ đào) phải cạnh tranh giải một bài toán toán học cực khó bằng sức mạnh phần cứng máy tính (tìm giá trị Nonce thích hợp). Người đầu tiên giải được sẽ giành quyền đóng gói khối và nhận phần thưởng. PoW bảo mật rất cao nhưng tiêu tốn tài nguyên điện năng khổng lồ và tốc độ xử lý giao dịch rất chậm (Bitcoin chỉ khoảng 7 TPS).
- **Proof of Stake (PoS - Bằng chứng cổ phần):** Thay thế việc khai thác bằng phần cứng bằng việc đặt cọc (stake) tiền điện tử. Hệ thống dựa vào tỷ lệ cổ phần đặt cọc để chọn ngẫu nhiên nút có quyền tạo khối mới. PoS tiết kiệm năng lượng hơn PoW, tốc độ nhanh hơn, nhưng có nguy cơ dẫn đến việc tích lũy tài sản (người giàu càng giàu và nắm quyền kiểm soát mạng).

2.4.3. Đồng thuận trong Private Blockchain (CFT và BFT)

Hệ thống chuỗi doanh nghiệp yêu cầu tốc độ xử lý cao và các nút tham gia đều được định danh rõ ràng. Do đó, họ áp dụng hai nhóm thuật toán tối ưu sau:

Đặc tính so sánh	Nhóm lỗi hỏng hóc (CFT - Crash Fault Tolerance)	Nhóm lỗi Byzantine (BFT - Byzantine Fault Tolerance)
Bản chất chống lỗi	Chỉ chịu được lỗi nút bị sập nguồn, mất mạng (đột ngột dừng hoạt động).	Chịu được cả lỗi nút bị sập VÀ nút cố tình gian lận, gửi dữ liệu sai.
Thuật toán đại diện	Raft, Paxos.	PBFT (Practical Byzantine Fault Tolerance), IBFT.
Cơ chế hoạt động	Bầu ra một nút Leader (Trưởng nhóm). Mọi giao dịch gửi tới Leader, Leader ghi logs và đồng bộ xuống các nút Follower. Nếu Leader sập, các nút còn lại bầu nhanh Leader mới.	Trải qua 3 giai đoạn đồng thuận nghiêm ngặt công khai: Pre-prepare, Prepare, và Commit thông qua việc trao đổi thông điệp toàn vẹn giữa tất cả các nút.
Khả năng chịu lỗi	Chịu lỗi tối đa lên tới $F = \frac{N-1}{2}$ (Mạng cần $2F + 1$ nút để hoạt động).	Chịu lỗi tối đa thấp hơn: $F = \frac{N-1}{3}$ (Mạng cần tối thiểu $3F + 1$ nút để hoạt động).
Ứng dụng doanh nghiệp	Thích hợp cho môi trường nội bộ doanh nghiệp tin cậy cao, cần tốc độ cực nhanh.	Thích hợp cho môi trường Consortium (liên minh nhiều bên đối tác có sự nghi kị lẫn nhau).

Bảng 1: So sánh thuật toán đồng thuận CFT và BFT

2.5. Hợp đồng thông minh (Smart Contract)

2.5.1. Khái niệm và đặc tính tất định (Determinism)

Hợp đồng thông minh là một chương trình máy tính (mã nguồn mã hóa) được triển khai trực tiếp và lưu trữ vĩnh viễn trên nền tảng Blockchain. Nó tự động thực thi các điều khoản thỏa thuận giữa các bên khi các điều kiện logic định trước được thỏa mãn mà không cần thông qua bên trung gian thứ ba.

Đặc tính tất định (Determinism): Đây là đặc tính bắt buộc tối thượng của Smart Contract. Với cùng một trạng thái sổ cái đầu vào (State input) và cùng một tham số truyền vào hàm, việc thực thi Smart Contract tại bất kỳ nút nào trong mạng lưới, tại bất kỳ thời điểm nào, cũng đều phải trả về một kết quả đầu ra giống nhau 100% và cập nhật trạng thái sổ cái đồng nhất. Do đó, các hàm lấy thời gian thực của máy cục bộ, hay hàm tạo số ngẫu nhiên thuần túy đều bị cấm sử dụng trực tiếp trong mã nguồn của Smart Contract.

2.5.2. Vòng đời thực thi của Hợp đồng thông minh

Vòng đời thực thi của một Hợp đồng thông minh trải qua 5 bước nghiêm ngặt:

1. **Viết mã (Coding):** Lập trình viên viết mã nguồn hợp đồng bằng ngôn ngữ chuyên dụng (như Solidity trên Ethereum, hoặc Go/Java/Node.js cho Chaincode trên Hyperledger Fabric).
2. **Biên dịch (Compilations):** Mã nguồn được biên dịch thành mã máy (Bytecode) để máy ảo (như EVM) hoặc môi trường thực thi (Docker container) có thể đọc hiểu.
3. **Triển khai (Deployment):** Bytecode được đóng gói vào một giao dịch đặc biệt và gửi lên mạng lưới Blockchain. Sau khi được các nút đồng thuận và đóng khối thành công, hợp đồng sẽ có một Địa chỉ duy nhất (Contract Address) cố định trên chuỗi.
4. **Định tuyến gọi hàm (Contract Router):** Khi người dùng hoặc ứng dụng client muốn tương tác, họ gửi một giao dịch đến địa chỉ của hợp đồng kèm theo tên hàm và tham số đầu vào. Hệ thống mạng lưới sẽ định tuyến (route) yêu cầu này đến đúng môi trường cô lập chứa mã hợp đồng để kích hoạt thực thi.
5. **Cập nhật trạng thái (State Update):** Sau khi tính toán xong, kết quả thực thi tạo ra một trạng thái mới (World State thay đổi). Trạng thái mới này được phân phối đồng bộ, xác nhận qua cơ chế đồng thuận mạng và ghi vĩnh viễn vào sổ cái phân tán.

2.6. Phân tích kiến trúc nền tảng Hyperledger Fabric

2.6.1. Mô hình Execute-Order-Validate

Các mạng Blockchain truyền thống như Bitcoin hay Ethereum sử dụng mô hình Order-Execute (Thứ tự trước - Thực thi sau). Nghĩa là các giao dịch được gom lại, sắp xếp thứ tự và đóng khối trước bởi các thợ đào (Order), sau đó tất cả các nút trong mạng đồng loạt tải khối về rồi mới tự chạy thực thi lại mã nguồn để cập nhật sổ cái (Execute). Mô hình này bộc lộ điểm nghẽn hiệu năng nghiêm ngặt (tất cả các nút đều phải chạy lại mọi thứ) và không đảm bảo tính bảo mật thương mại.

Hyperledger Fabric kiến tạo một kiến trúc đột phá hoàn toàn: Execute-Order-Validate (Thực thi trước - Sắp xếp sau - Kiểm tra lại). Quy trình gồm 3 bước riêng biệt phân rã nhiệm vụ cho từng loại nút chuyên biệt:

- **Execute (Thực thi/Ký duyệt):** Ứng dụng gửi đề xuất giao dịch đến các nút ký duyệt chỉ định trước (Endorsing Nodes). Các nút này chạy thử nghiệm mã Smart Contract (gọi là Chaincode) một cách độc lập trong môi trường bảo mật (Docker) để tạo ra kết quả đọc/ghi (Read-Write Set) và ký số xác nhận mà không ghi ngay vào sổ cái.
- **Order (Sắp xếp thứ tự):** Ứng dụng thu thập đủ chữ ký phê duyệt từ bước 1 và gửi Read-Write Set đến dịch vụ sắp xếp (Ordering Service / Orderer Nodes). Các nút Orderer chịu trách nhiệm duy nhất là sắp xếp các giao dịch theo đúng trình tự thời gian và đóng gói chúng thành các khối (Blocks) mà không quan tâm bên trong giao dịch chạy logic gì. Điều này tối ưu hóa băng thông mạng cực lớn.
- **Validate (Xác thực lại):** Khối từ Orderer được phát tán tới các nút lưu trữ (Committing Nodes). Các nút này kiểm tra lại xem các chữ ký phê duyệt có hợp lệ và tuân thủ chính sách hay không, đồng thời kiểm tra xem có xung đột dữ liệu Read-Write (xung đột song song) hay không. Nếu hợp lệ, giao dịch chính thức được ghi (Commit) và cập nhật vào cơ sở dữ liệu trạng thái thế giới (World State).

2.6.2. Vấn đề thực tiễn và bài toán tối ưu tài nguyên

Mặc dù kiến trúc Execute-Order-Validate mang lại hiệu năng cao và phân quyền mạnh mẽ cho doanh nghiệp lớn, tuy nhiên, việc vận hành và triển khai một mạng Hyperledger Fabric chuẩn hóa trong thực tế đối diện với sự phức tạp và công kênh rất lớn về mặt kiến trúc tài nguyên phần cứng hệ thống:

- **Hệ thống CA (Certificate Authority) đồ sộ:** Fabric quản lý định danh người dùng và các nút bằng chứng thư số X.509. Mỗi tổ chức (Organization) tham gia yêu cầu phải duy trì một hoặc nhiều cụm máy chủ CA độc lập, gây quá tải quản trị.
- **Cụm Orderer phức tạp:** Để đảm bảo tính sẵn sàng cao, dịch vụ Orderer phải chạy thuật toán Raft gồm nhiều nút cấu hình phức tạp, đòi hỏi cấu hình mạng ổn định và độ trễ cực thấp.
- **Cơ sở dữ liệu CouchDB/LevelDB phân tách:** Mỗi nút Peer phải gánh một cơ sở dữ liệu riêng để lưu trữ World State (thường là CouchDB để hỗ trợ truy vấn JSON nâng cao). Việc đồng bộ hóa dữ liệu liên tục giữa các container CouchDB tiêu tốn lượng RAM và I/O đĩa cứng cực kỳ lớn.
- **Rào cản doanh nghiệp vừa và nhỏ (SME):** Sự phức tạp của việc thiết lập hàng chục container Docker chạy đồng thời (gồm các Peer, Orderer, CA, CouchDB) đòi hỏi hạ tầng máy chủ mạnh mẽ và đội ngũ DevOps trình độ chuyên môn cao. Đối với các doanh nghiệp vừa và nhỏ (SME) hoặc các dự án có nguồn lực tài nguyên hạn chế, việc cấu hình và duy trì một hệ thống Fabric đầy đủ cấu hình tiêu chuẩn là một "gánh nặng chi phí" và lãng phí năng lực tính toán.

Chương 3: Phân tích và Đặc tả yêu cầu hệ thống

3.1. Kiến trúc tổng quát của Khoai Chain

3.1.1. Các thay đổi chính

Để giải quyết bài toán công kênh của các nền tảng blockchain công nghiệp, Khoai Chain được thiết kế với triết lý tinh gọn (minimalist), tập trung vào cốt lõi của sổ cái phân tán và loại bỏ các thành phần không cần thiết cho quy mô doanh nghiệp vừa và nhỏ. Các thay đổi mang tính bước ngoặt bao gồm:

- **Tích hợp vai trò Node (Node Consolidation):** Thay vì phân rã hệ thống thành các nút độc lập (như Orderer Node để sắp xếp, Peer Node để lưu trữ, và CA để cấp phép), Khoai Chain tích hợp tất cả các chức năng đồng thuận, định tuyến giao dịch và lưu trữ vào một module Node duy nhất. Điều này giúp giảm thiểu độ trễ giao tiếp qua mạng lưới và tiết kiệm tối đa RAM/CPU.
- **Cơ chế lưu trữ nhúng (Embedded Storage):** Thay vì yêu cầu thiết lập các container cơ sở dữ liệu bên ngoài (như CouchDB), Khoai Chain sử dụng cơ sở dữ liệu dạng Key-Value được nhúng trực tiếp vào lõi hệ thống. Quá trình đồng bộ trạng thái sổ cái diễn ra nội bộ, loại bỏ hoàn toàn các cấu hình kết nối mạng phức tạp giữa Core và Database.
- **Tách biệt hoàn toàn SDK Hợp đồng thông minh:** Mã nguồn thực thi (Smart Contract) được tách ra khỏi lõi mạng P2P thông qua bộ công cụ `khoaichain-sdk`. Sự thay đổi này giúp các nhà phát triển và sinh viên chỉ cần kế thừa các interface có sẵn để viết logic nghiệp vụ, biên dịch và nạp vào mạng lưới mà không cần hiểu sâu về kiến trúc lõi của Khoai Chain.
- **Lược bỏ cơ chế kinh tế (Tokenomics):** Do hệ thống phục vụ chuỗi doanh nghiệp nội bộ đã có sự tin tưởng và định danh nhất định, toàn bộ cơ chế khai thác (Mining), phí giao dịch (Gas fee) và phần thưởng khối đều được loại bỏ để tăng tốc độ đóng gói giao dịch.

3.1.2. So sánh với Hyperledger Fabric

Sự khác biệt cốt lõi giữa Khoai Chain và tiêu chuẩn công nghiệp Hyperledger Fabric được thể hiện chi tiết qua bảng đối chiếu sau:

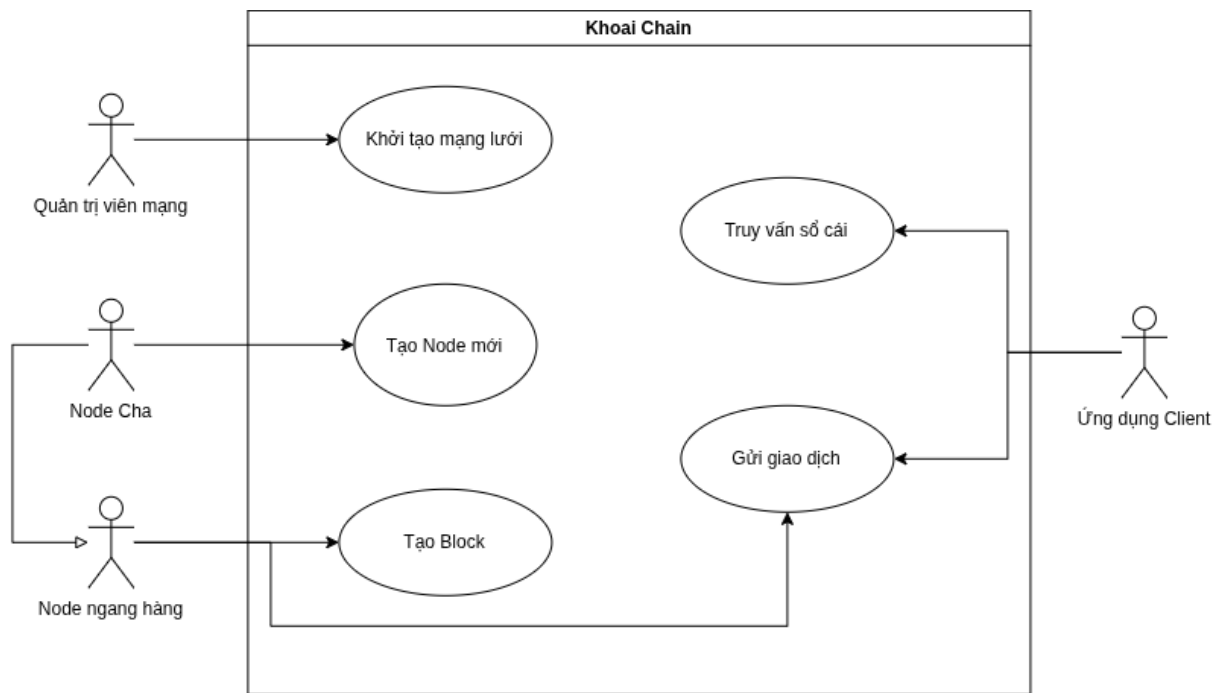
Tiêu chí	Hyperledger Fabric	Khoai Chain
Kiến trúc Node	Phân rã phức tạp (Peer, Orderer, CA hoạt động trên các vùng nhớ riêng).	Tích hợp (Tất cả vai trò xử lý, đồng thuận, lưu trữ nằm trong một tiến trình duy nhất).
Lưu trữ dữ liệu	Cần hệ quản trị CSDL độc lập (CouchDB/LevelDB chạy khác container).	Tích hợp CSDL Key-Value nhúng trực tiếp vào Core (Tiết kiệm I/O).
Phát triển Contract	Chaincode chạy trong môi trường Docker bị cô lập, cấu hình giao tiếp mạng phức tạp.	Phát triển qua <code>khoaichain-sdk</code> độc lập, dễ dàng biên dịch và thực thi trực tiếp.
Quản lý định danh	Sử dụng Certificate Authority (CA) với chứng chỉ X.509, yêu cầu duy trì máy chủ CA riêng.	Quản lý tinh gọn qua tệp cấu hình tĩnh (yaml) và cơ chế khóa xác thực nội bộ.
Mức độ triển khai	Rất nặng. Yêu cầu kiến thức DevOps sâu rộng, tốn nhiều cấu hình YAML/Docker Compose.	Rất nhẹ. Khởi chạy thông qua CLI framework đơn giản hoặc một container duy nhất.
Đối tượng phù hợp	Tập đoàn lớn, liên minh đa quốc gia, ngân hàng.	Sinh viên nghiên cứu, doanh nghiệp vừa và nhỏ (SME), hệ thống thử nghiệm cục bộ.

Bảng 2: So sánh đặc tính kỹ thuật giữa Hyperledger Fabric và Khoai Chain

3.2. Mô hình các chức năng

Sơ đồ Use Case tổng quát mô tả các tương tác cốt lõi giữa người dùng và hệ thống lõi của mạng lưới. Trong hệ thống này, các tác nhân được chia làm hai nhóm chính: nhóm quản trị hạ tầng (bao gồm Quản trị viên mạng, Node cha, Node ngang hàng) và nhóm người dùng cuối (Ứng dụng Client).

Cấu trúc phân quyền được thiết kế chặt chẽ nhằm đảm bảo tính phi tập trung nhưng vẫn kiểm soát được vòng đời sinh trưởng của các nút trong mạng. Chi tiết về luồng thực thi và các ràng buộc của từng chức năng được đặc tả cụ thể trong phần dưới đây.



Hình 1: Sơ đồ Use Case mạng lưới Khoai Chain

a) Khởi tạo mạng lưới

Thuộc tính	Mô tả chi tiết
Tên Use Case	Khởi tạo mạng lưới (Khởi tạo khối Genesis)
Tác nhân	Quản trị viên mạng
Tiền điều kiện	Hệ thống máy chủ được cài đặt mã nguồn lõi Khoai Chain. Chưa có dữ liệu sổ cái nào tồn tại.
Luồng sự kiện chính	1. Quản trị viên cấu hình các tham số mạng cơ bản (thời gian block, thuật toán đồng thuận) vào file cấu hình. 2. Quản trị viên chạy lệnh khởi tạo trên CLI. 3. Hệ thống tạo ra Khối Genesis (Khối số 0) chứa định danh của Quản trị viên. 4. Hệ thống ghi Khối Genesis vào cơ sở dữ liệu cục bộ và mở cổng P2P.
Luồng ngoại lệ	- Nếu định dạng file cấu hình sai: Hệ thống báo lỗi và dừng tiến trình. - Nếu cổng P2P đã bị chiếm dụng: Hệ thống báo lỗi không thể mở kết nối mạng.
Hậu điều kiện	Mạng lưới được khởi động với 1 Nút gốc duy nhất. Sổ cái ở trạng thái sẵn sàng tiếp nhận Node mới.

Bảng 3: Đặc tả chi tiết Use Case Khởi tạo mạng lưới Khoai Chain

b) Tạo Node mới

Thuộc tính	Mô tả chi tiết
Tên Use Case	Tạo Node mới
Tác nhân	Quản trị viên mạng, Node cha (Node quản lý)
Tiền điều kiện	Mạng lưới đang hoạt động. Tác nhân phải có quyền hợp lệ trên chuỗi để thực hiện cấp phép.
Luồng sự kiện chính	1. Tác nhân gửi yêu cầu tạo Node con thông qua SDK, kèm theo địa chỉ Public Key của Node mới. 2. Hệ thống kiểm tra chữ ký và cấp bậc (Tier) của Tác nhân trong Hợp đồng thông minh. 3. Hệ thống (bao gồm các Nút xác thực) thực hiện đồng thuận để ghi giao dịch cấp phép vào sổ cái. 4. Nút mới khởi động, kết nối tới Node cha và bắt đầu đồng bộ dữ liệu.
Luồng ngoại lệ	- Tác nhân không đủ quyền (không phải Node cha): Giao dịch bị từ chối, trả về lỗi "Unauthorized". - Mạng lưới không đạt đồng thuận: Giao dịch cấp phép bị hủy bỏ.
Hậu điều kiện	Node mới được định danh hợp lệ trên sổ cái và có thể tham gia vào mạng lưới theo đúng cấp bậc được giao.

Bảng 4: Đặc tả chi tiết Use Case Tạo Node mới Khoai Chain

c) Tạo Block (Đồng thuận khối)

Thuộc tính	Mô tả chi tiết
Tên Use Case	Tạo Block (Đồng thuận khối)
Tác nhân	Nút xác thực (Validator Node)
Tiền điều kiện	Có ít nhất 1 giao dịch hợp lệ đang chờ trong Mempool. Nút đang ở trạng thái hoạt động.
Luồng sự kiện chính	1. Nút đến lượt làm Trưởng nhóm (Leader) lấy các giao dịch từ Mempool. 2. Nút Trưởng nhóm đóng gói giao dịch thành Khối đề xuất và phát tán cho các Nút xác thực khác. 3. Các Nút xác thực kiểm tra tính hợp lệ của Khối và phát tín hiệu bỏ phiếu (Vote). 4. Khi đủ tỷ lệ phiếu đồng ý ($> 51\%$), Khối được chốt.
Luồng ngoại lệ	- Khối chứa giao dịch sai logic: Các nút từ chối bỏ phiếu, Khối bị hủy sau thời gian timeout. - Nút Trưởng nhóm mất kết nối: Mạng lưới tự động bầu Trưởng nhóm mới.
Hậu điều kiện	Khối mới được nối vào chuỗi. Trạng thái cơ sở dữ liệu State DB được cập nhật.

Bảng 5: Đặc tả chi tiết Use Case Tạo Block (Đồng thuận khối)

d) Gửi giao dịch

Thuộc tính	Mô tả chi tiết
Tên Use Case	Gửi giao dịch
Tác nhân	Ứng dụng Client
Tiền điều kiện	Client có khóa bí mật (Private Key) hợp lệ. Kết nối mạng tới ít nhất 1 Nút đang mở.
Luồng sự kiện chính	1. Client tạo dữ liệu giao dịch nghiệp vụ (ví dụ: tạo lô thuốc) và ký số bằng Private Key. 2. Client gửi gói tin giao dịch qua API/RPC tới Nút tiếp nhận. 3. Nút tiếp nhận xác thực sơ bộ (chữ ký, định dạng) và đẩy vào Mempool. 4. Nút tiếp nhận trả về mã băm (TxHash) cho Client để theo dõi.
Luồng ngoại lệ	- Chữ ký không hợp lệ: Nút tiếp nhận trả về lỗi 400 Bad Request ngay lập tức. - Mempool đang đầy: Nút trả về lỗi 429 Too Many Requests.
Hậu điều kiện	Giao dịch nằm trong Mempool, chờ được đóng gói vào Khối.

Bảng 6: Đặc tả chi tiết Use Case Gửi giao dịch

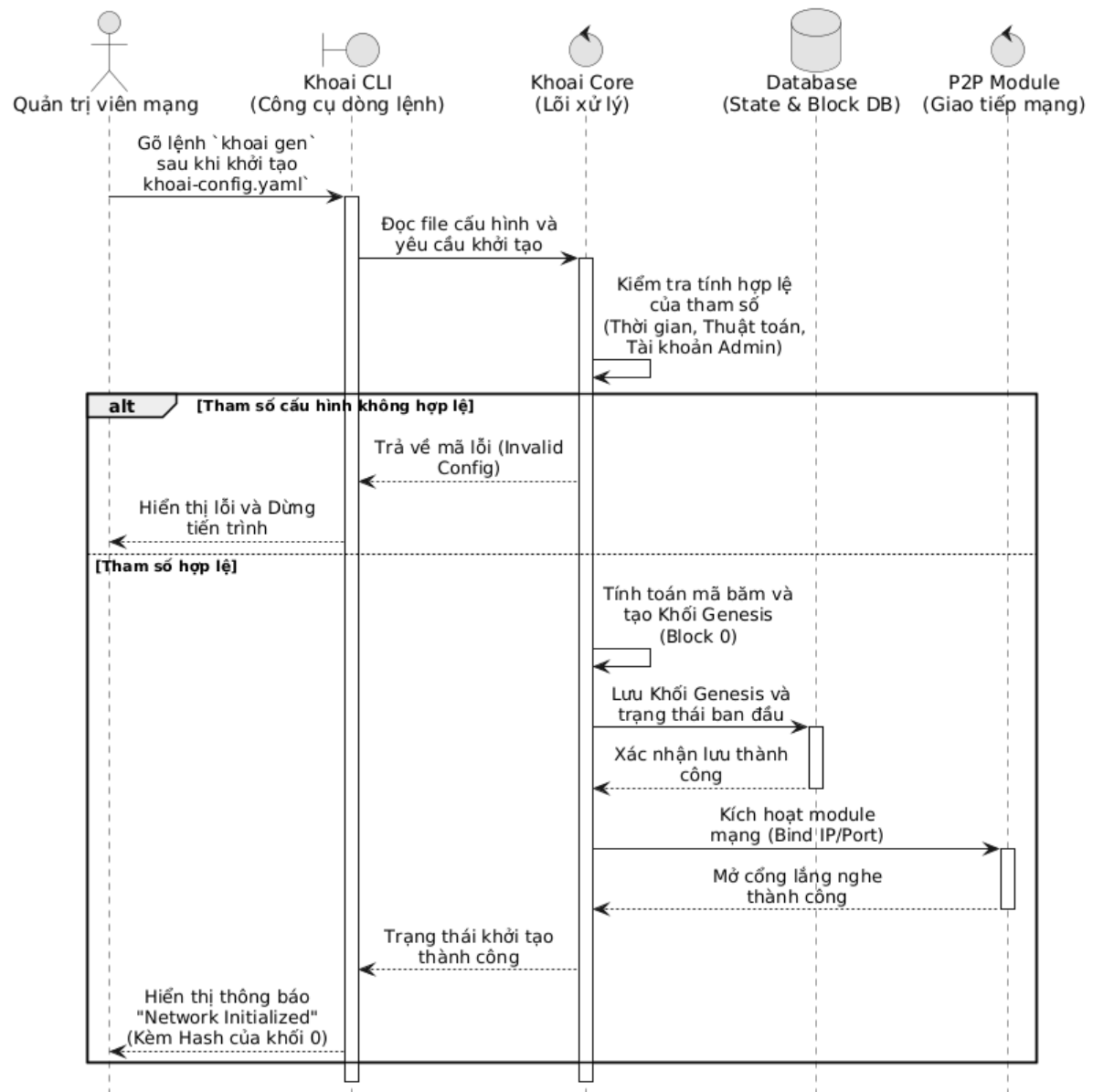
e) Truy vấn số cái

Thuộc tính	Mô tả chi tiết
Tên Use Case	Truy vấn số cái
Tác nhân	Ứng dụng Client
Tiền điều kiện	Client kết nối thành công tới Nút trong mạng lưới.
Luồng sự kiện chính	1. Client gửi yêu cầu đọc dữ liệu (ví dụ: truy xuất thông tin lô thuốc) tới Nút. 2. Nút truy xuất dữ liệu từ State DB cục bộ của mình. 3. Nút định dạng dữ liệu thành JSON và trả về cho Client.
Luồng ngoại lệ	- Dữ liệu không tồn tại: Nút trả về phản hồi rỗng hoặc thông báo "Not Found". - Yêu cầu truy vấn quá lớn (quét toàn bộ chuỗi): Nút ngắt kết nối để tránh quá tải.
Hậu điều kiện	Client nhận được dữ liệu trạng thái mới nhất từ số cái.

Bảng 7: Đặc tả chi tiết Use Case Truy vấn số cái

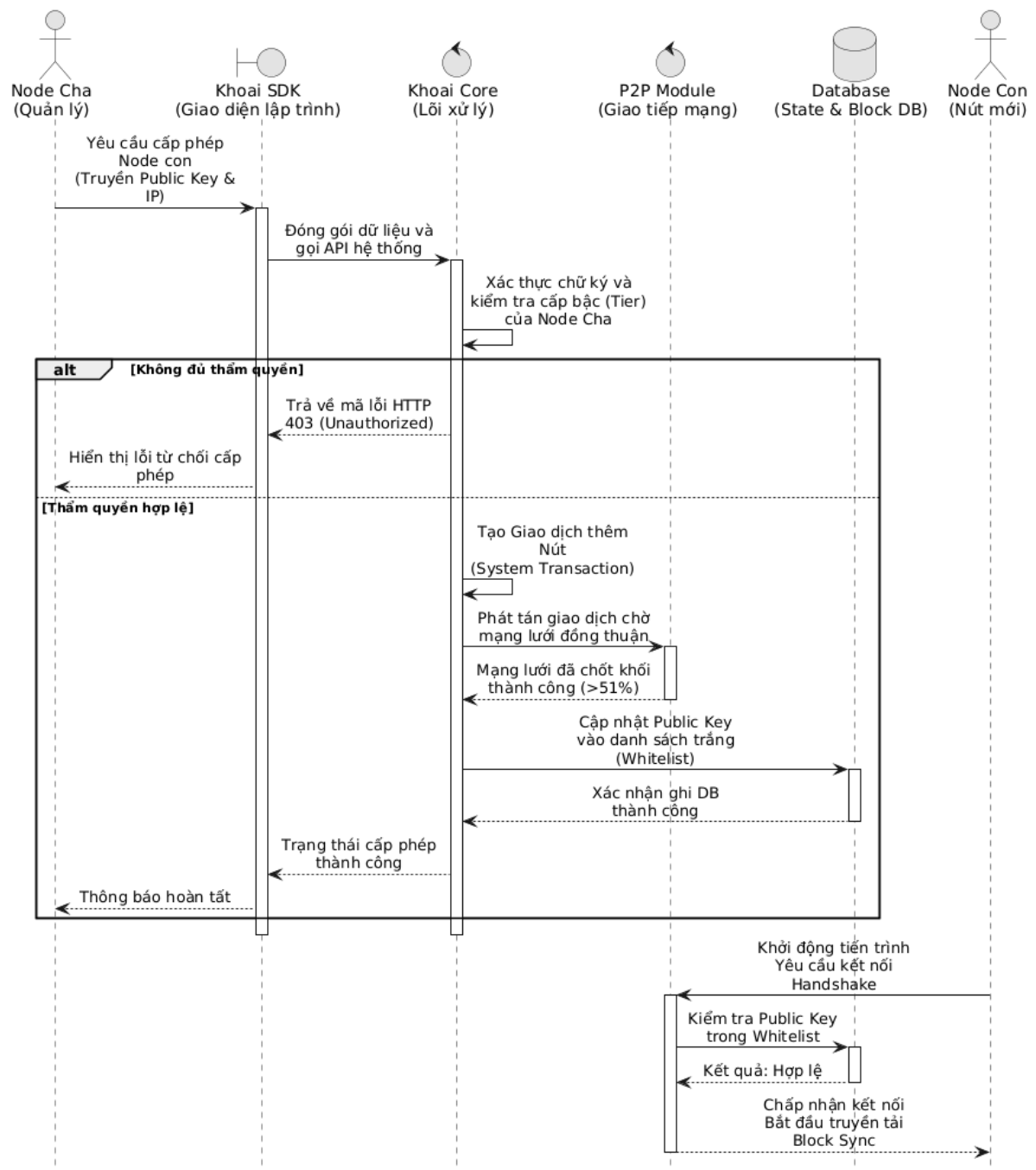
3.3. Các trường hợp sử dụng

a) Khởi tạo mạng lưới



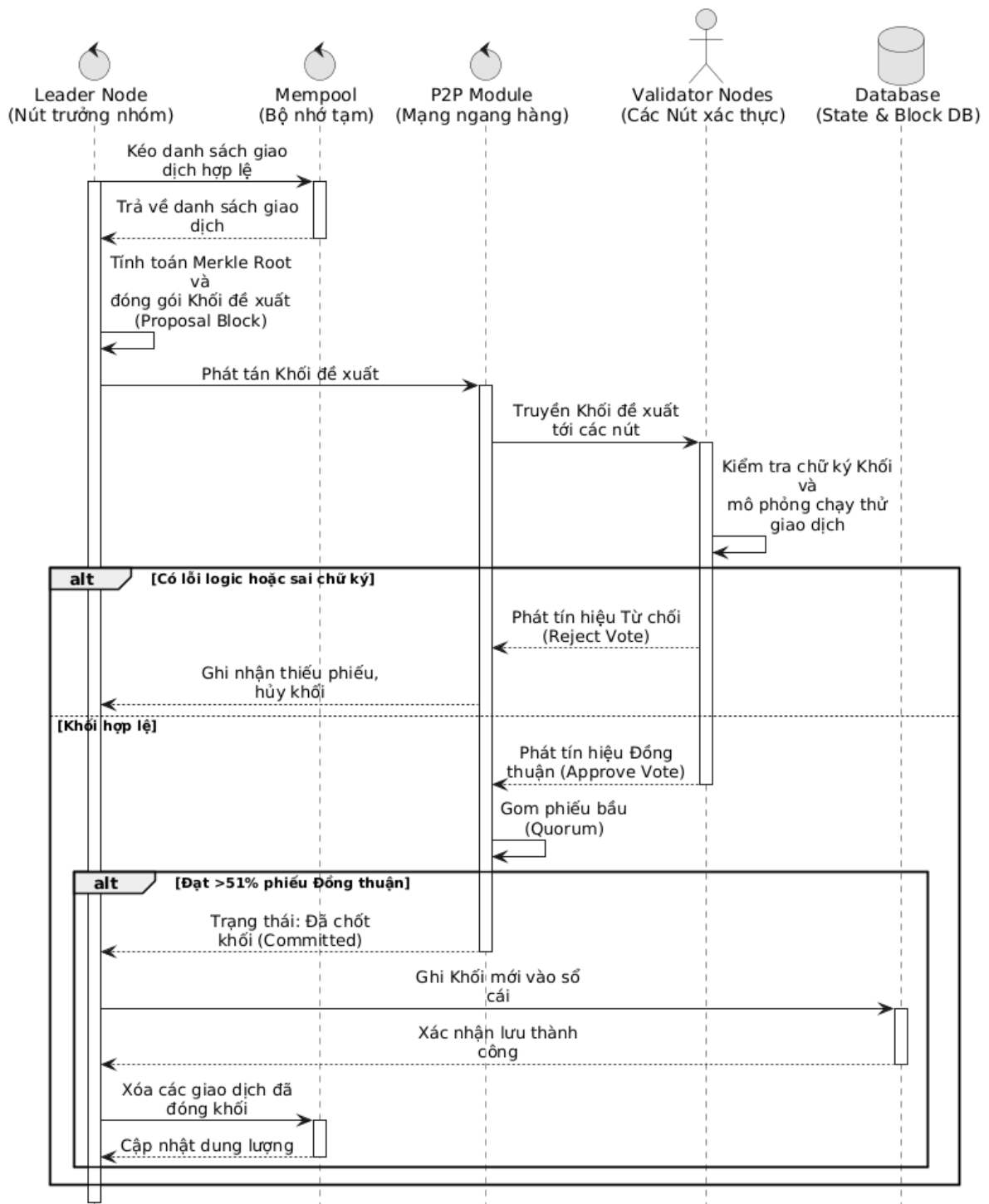
Hình 2: Sơ đồ Sequence khởi tạo mạng lưới Khoai Chain

b) Tạo Node mới



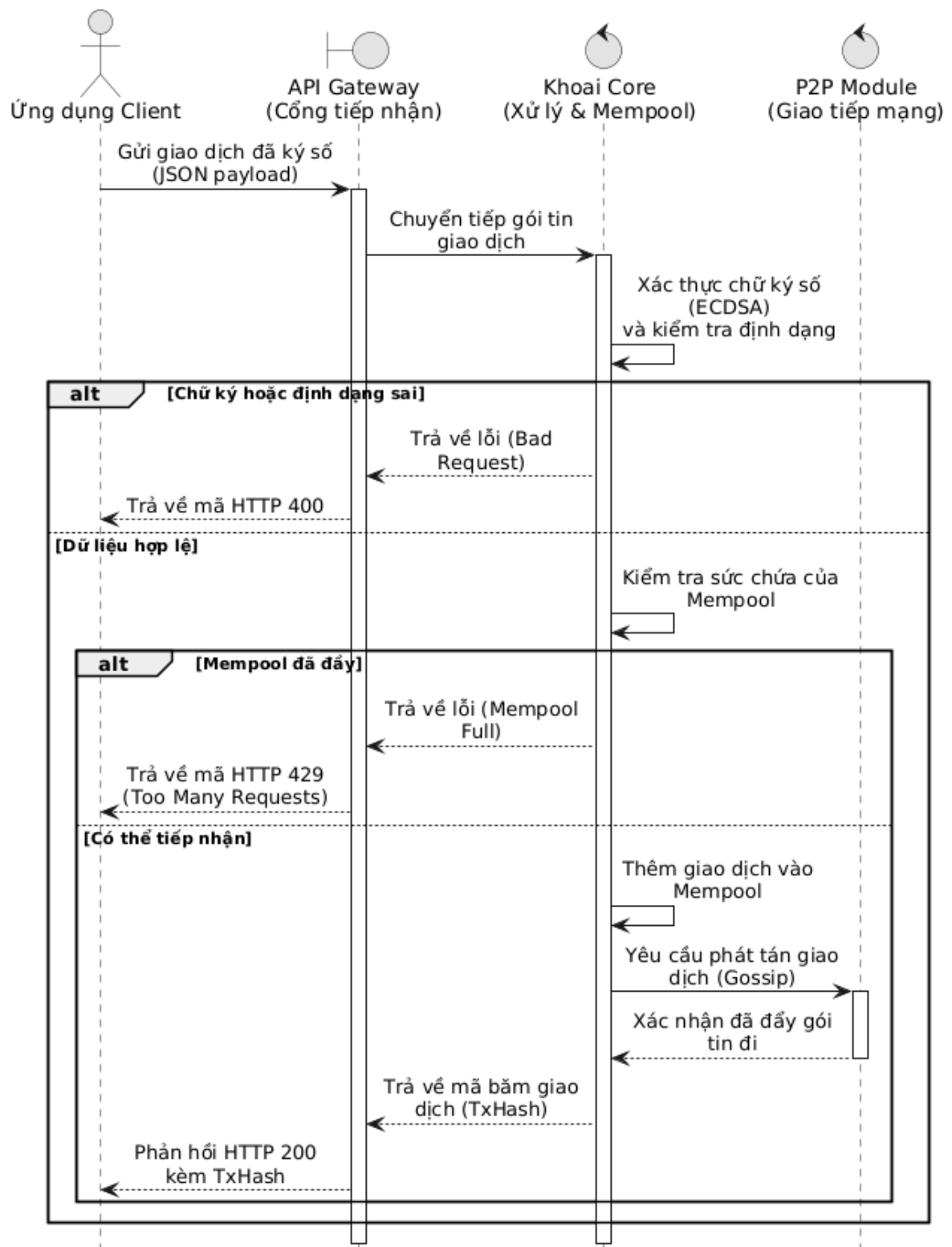
Hình 3: Sơ đồ Sequence tạo Node mới Khoai Chain

c) Tạo Block



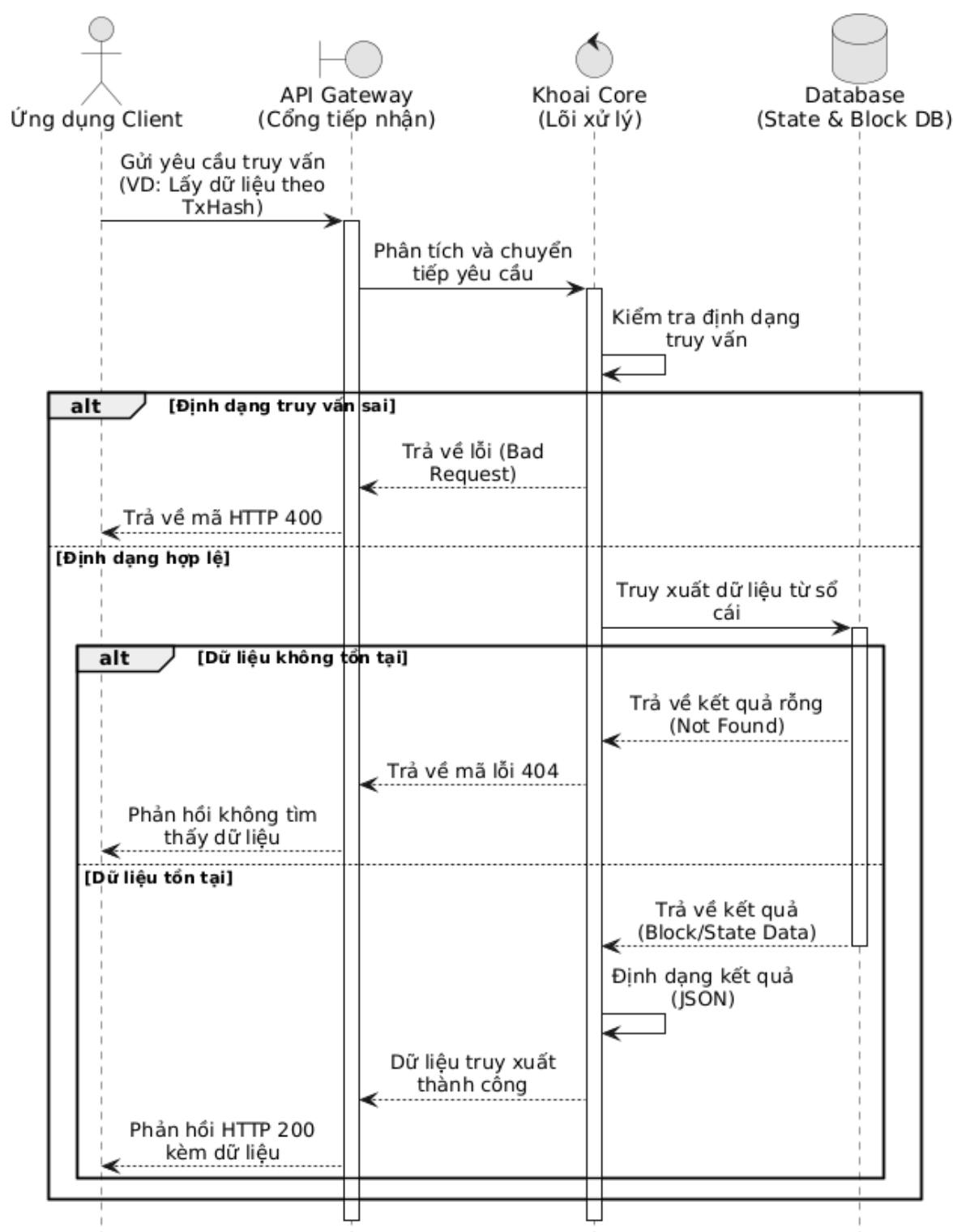
Hình 4: Sơ đồ Sequence tạo Block Khoai Chain

d) Gửi giao dịch



Hình 5: Sơ đồ Sequence gửi giao dịch Khoai Chain

e) Truy vấn sổ cái



Hình 6: Sơ đồ Sequence truy vấn sổ cái Khoai Chain

3.4. Yêu cầu phi chức năng

Chương 4: Thiết kế Kiến trúc Khoai Chain

4.1. Kiến trúc tổng thể hệ thống

4.2. Thiết kế Module P2P (Peer, Server, Handler, Message)

4.3. Thiết kế Core Blockchain

4.3.1. Cấu trúc dữ liệu Block và Transaction

4.3.2. Luồng hoạt động của Mempool

4.4. Thiết kế hệ thống Smart Contract (Manager, Router, Interface)

4.5. Thiết kế Cấu trúc lưu trữ dữ liệu (Database Storage)

Chương 5: Hướng dẫn sử dụng và Triển khai

- 5.1. Khởi tạo môi trường và cấu hình (khoai-config.yaml, get-env)
- 5.2. Chạy thử nghiệm bằng CLI và Docker
- 5.3. Kịch bản chạy thử nghiệm mẫu cho sinh viên
- 5.4. Các chức năng đang phát triển và Hạn chế hiện tại

Kết luận

Tóm tắt các chức năng đã chốt

Định hướng hoàn thiện mã nguồn